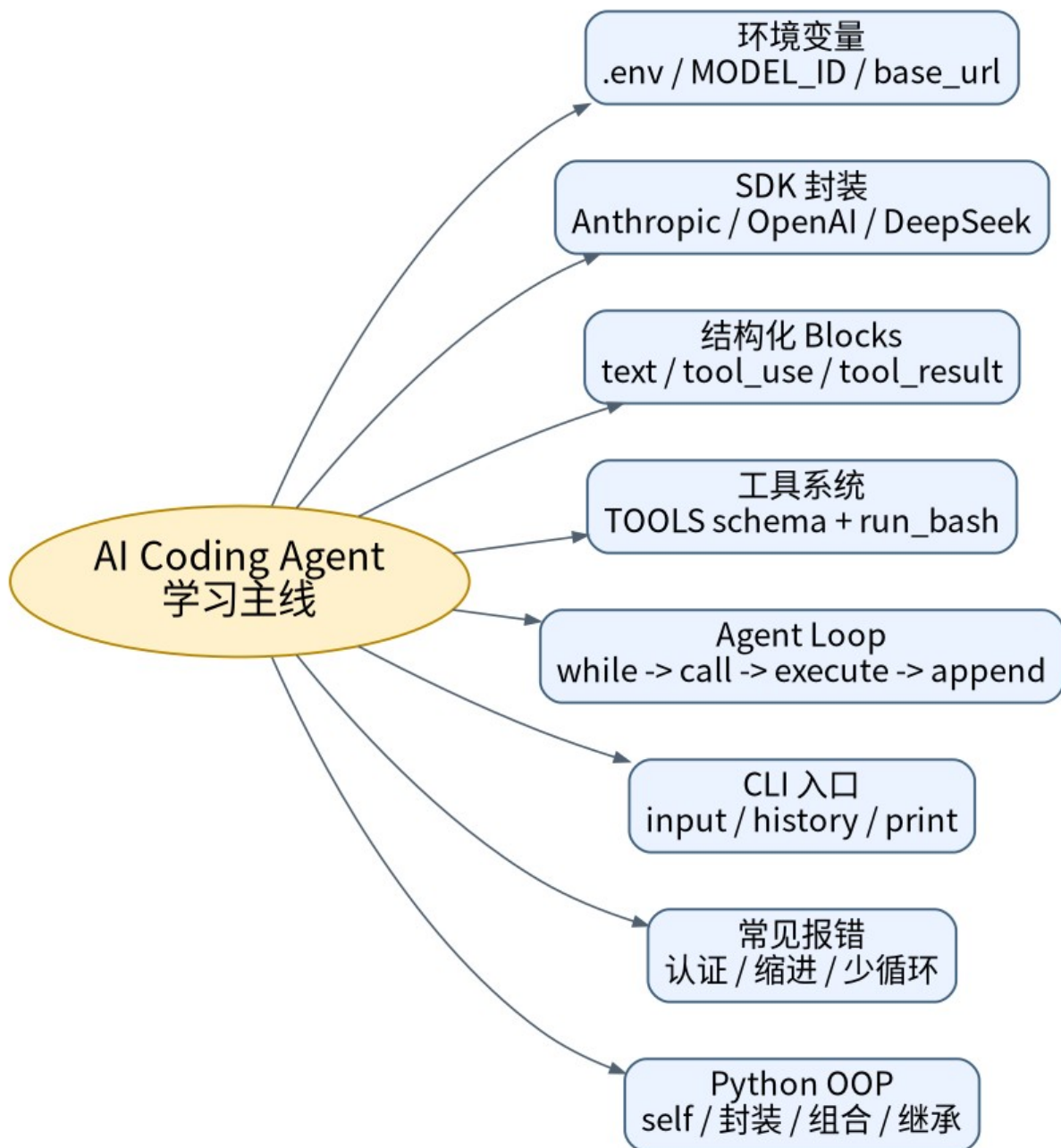


AI Coding Agent 学习笔记

主题：Anthropic SDK、DeepSeek 兼容接口、Tool Calling、Agent Loop 与 Python 封装

本笔记定位

这是对本次对话的归纳整理，不是 API 官方文档。重点放在：代码如何运行、为什么报错、如何把 LLM API 封装成统一的 agent runtime。



思维导图 1：本次对话总览

目录

1. 核心结论速览
2. 环境变量与客户端初始化
3. Anthropic SDK、DeepSeek 与 OpenAI 风格 API
4. 工具定义 TOOLS 与 JSON Schema
5. run_bash 工具执行器
6. Agent Loop 核心循环
7. CLI 入口与 history 管理
8. Block / getattr / block.text 的理解
9. 常见报错与定位
10. SDK 封装层与结构化 Block 返回
11. Python 封装、组合、继承与 self.http
12. 推荐修正版与工程化方向

1. 核心结论速览

重点 1：SDK 不等于模型
from anthropic import Anthropic 代表你使用 Anthropic SDK 的调用格式；真正请求到哪里，由 base_url 决定；真正用哪个模型，由 provider endpoint + model id 共同决定。

重点 2：Agent Loop 的本质
模型不是直接“执行命令”，而是返回 tool_use block。你的 Python runtime 读取 tool_use，执行 run_bash，再把 tool_result 追加回 messages，循环直到模型停止调用工具。

重点 3：第二段代码不能用的主要原因
第一，第二段代码 pop 掉了 ANTHROPIC_API_KEY，导致 SDK 找不到认证方式。第二，第二段 agent_loop 少了 while True，工具执行结果不会再喂回模型生成最终回复。

问题	正确理解
.env 写 DeepSeek key，为什么还用 Anthropic SDK?	因为可以通过 ANTHROPIC_BASE_URL 指向 DeepSeek 的 Anthropic-compatible endpoint。
模型为什么说自己是 Claude?	协议像 Claude，加上 system prompt 没强制身份，模型可能沿用 Claude 风格自我描述。
为什么不能 print(block.content)?	不同 block 字段不同：TextBlock 用 .text，ToolUseBlock 用 .input，tool_result 才常见 content。
self.http 是继承来的吗?	不是。self.http = http_client 是组合/依赖注入，不是继承。

2. 环境变量与客户端初始化

相关代码：

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv(override=True)

if os.getenv("ANTHROPIC_BASE_URL"):
    os.environ.pop("ANTHROPIC_AUTH_TOKEN", None)

client = Anthropic(base_url=os.getenv("ANTHROPIC_BASE_URL"))
MODEL = os.environ["MODEL_ID"]
```

代码/变量	作用	注意点
load_dotenv(override=True)	读取 .env 并写入 os.environ	override=True 表示 .env 覆盖已有环境变量
ANTHROPIC_BASE_URL	改变 SDK 请求的 API 地址	例如 https://api.deepseek.com/anthropic
MODEL_ID	模型名称字符串	只是字符串；能不能用取决于 endpoint 是否识别
ANTHROPIC_API_KEY	Anthropic SDK 默认寻找的 API key	如果被 pop 删除，就可能认证失败
ANTHROPIC_AUTH_TOKEN	另一类认证变量	原代码删除它是为了避免某些兼容接口冲突

重点：不要把 API key 删错

第一段删除的是 ANTHROPIC_AUTH_TOKEN；第二段删除的是 ANTHROPIC_API_KEY。你用 Anthropic SDK 时，SDK 至少需要 api_key/auth_token/credentials 之一。删掉 ANTHROPIC_API_KEY 后，常见报错就是 Could not resolve authentication method。

更稳的写法：

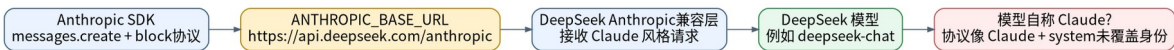
```
client = Anthropic(
    api_key=os.getenv("DEEPSEEK_API_KEY") or os.getenv("ANTHROPIC_API_KEY"),
    base_url=os.getenv("ANTHROPIC_BASE_URL")
)
```

3. Anthropic SDK、DeepSeek 与 OpenAI 风格 API

调用方式	典型代码	输出结构	适用对象
Anthropic SDK	client.messages.create(..)	content = [TextBlock, ToolUseBlock, ...]	Claude 或 Anthropic-compatible endpoint
OpenAI SDK	client.chat.completions.create(...)	choices[0].message / tool_calls	OpenAI、DeepSeek 原生 OpenAI-compatible API
DeepSeek Anthropic 兼容层	Anthropic(base_url=".../anthropic")	Claude 风格 blocks	想用 DeepSeek 但保留 Anthropic block 体验

重点：协议 ≠ 模型身份

Anthropic SDK 是客户端封装；DeepSeek /anthropic 是服务端兼容层；模型是否 DeepSeek，看请求实际打到哪个 endpoint 和 model id，而不是 import 名字。



思维导图 2: DeepSeek Anthropic-compatible 模式与身份错觉

4. 工具定义 TOOLS 与 JSON Schema

```
TOOLS = [{
    "name": "bash",
    "description": "Run a shell command.",
    "input_schema": {
        "type": "object",
        "properties": {"command": {"type": "string"}},
        "required": ["command"],
    },
}]
```

这段不是普通“字符串堆叠”，而是一个工具说明书。它告诉模型：有一个叫 bash 的工具；这个工具接收一个对象；对象里必须有 command 字段；command 必须是字符串。

字段	含义
name	工具名。模型想调用工具时会引用这个名字。
description	给模型看的能力说明，用于判断什么时候该调用该工具。
input_schema	JSON Schema，约束工具输入格式。
required	指定必填字段，防止模型漏传 command。

重点：Tool schema 是给模型看的 API 合约

LLM 本身不会真的执行 bash。它只会生成 tool_use 请求。真正执行命令的是你的 Python 函数 run_bash。

5. run_bash 工具执行器

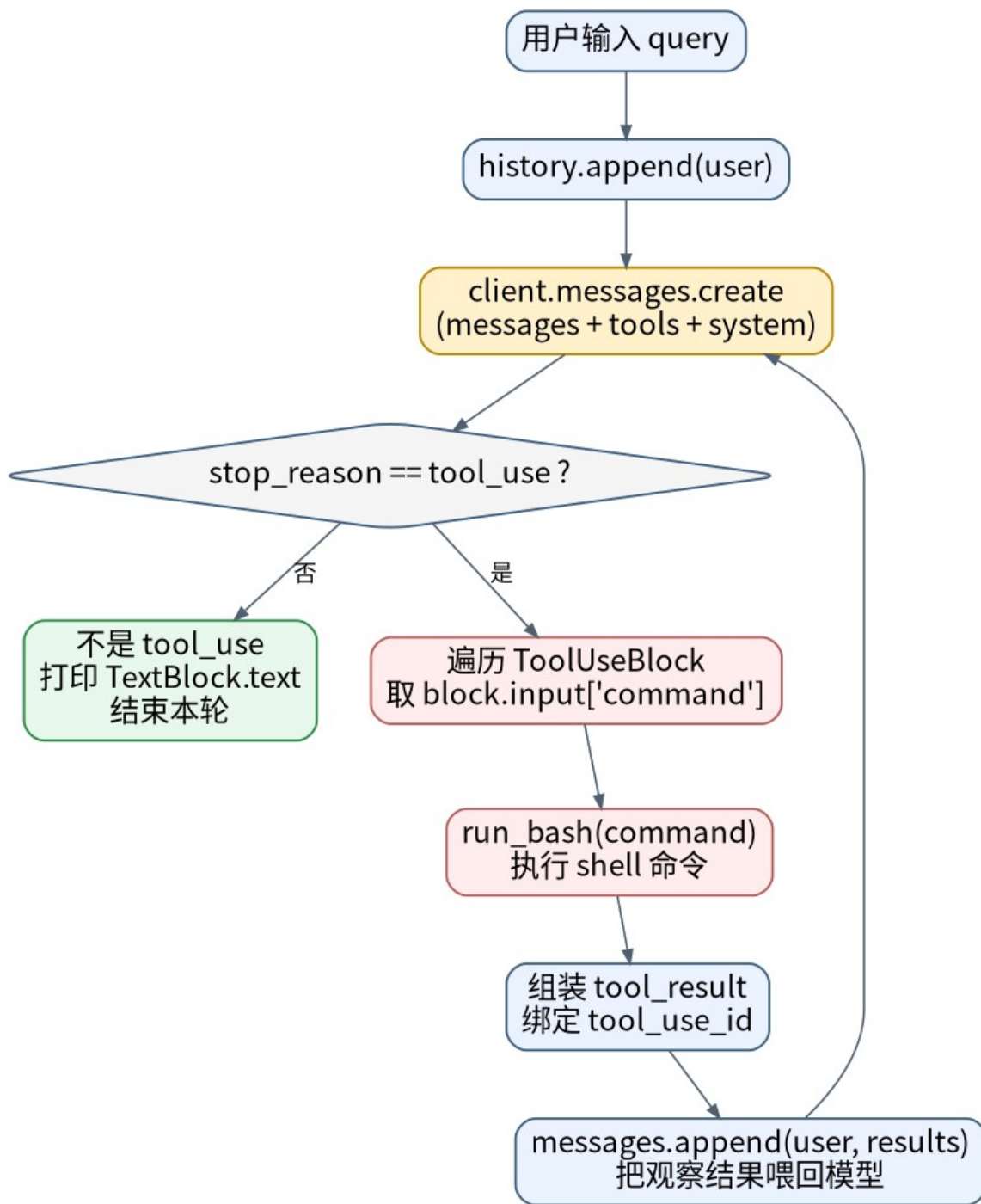
```
def run_bash(command: str) -> str:
    dangerous = ["rm -rf /", "sudo", "shutdown", "reboot", "> /dev/"]
    if any(d in command for d in dangerous):
        return "Error: Dangerous command blocked"
    try:
        r = subprocess.run(command, shell=True, cwd=os.getcwd(),
                           capture_output=True, text=True, timeout=120)
        out = (r.stdout + r.stderr).strip()
        return out[:50000] if out else "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: Timeout (120s)"
    except (FileNotFoundError, OSError) as e:
        return f"Error: {e}"
```

代码点	解释
dangerous 列表	黑名单式危险命令拦截。只能做基础防护，不是完整安全方案。
any(d in command for d in dangerous)	只要 command 包含任意危险片段，就返回错误。
subprocess.run(..., shell=True)	让系统 shell 执行字符串命令；灵活但有注入风险。
cwd=os.getcwd()	在当前工作目录执行命令。
capture_output=True	捕获 stdout 和 stderr。
text=True	返回字符串而不是 bytes。
timeout=120	防止命令卡死或长时间运行。
out[:50000]	限制返回长度，避免输出过大。

安全重点

黑名单防护很容易被绕过。工程级 agent 更推荐白名单、sandbox、容器隔离、文件权限控制、命令审计和人工确认机制。

6. Agent Loop 核心循环



思维导图 3: Agent Loop 执行闭环

```
def agent_loop(messages: list):
    while True:
        response = client.messages.create(
            model=MODEL, system=SYSTEM, messages=messages,
            tools=TOOLS, max_tokens=8000,
        )
```

```

messages.append({"role": "assistant", "content": response.content})

if response.stop_reason != "tool_use":
    return

results = []
for block in response.content:
    if block.type == "tool_use":
        output = run_bash(block.input["command"])
        results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": output,
        })

messages.append({"role": "user", "content": results})

```

步骤	作用
1. messages.create	把当前对话历史、工具列表、系统提示词发给模型。
2. append assistant	保存模型这次的输出，包含 text block 或 tool_use block。
3. 判断 stop_reason	如果不是 tool_use，说明模型不想再调用工具，本轮结束。
4. 执行工具	找到 ToolUseBlock，读取 command，调用 run_bash。
5. 组装 tool_result	用 tool_use_id 绑定结果属于哪次工具调用。
6. append user results	按协议把工具结果作为 user message 喂回模型。

重点：while True 不能省

如果省掉 while True，工具结果虽然被 append 进 history，但不会再次发给模型。结果就是：命令可能执行了，却没有最终文字回答。

7. CLI 入口与 history 管理

```
if __name__ == "__main__":
    history = []
    while True:
        query = input(">>> ")
        if query.strip().lower() in ("q", "exit", ""):
            break
        history.append({"role": "user", "content": query})
        agent_loop(history)
        response_content = history[-1]["content"]
```

代码点	解释
if __name__ == "__main__"	只有当前文件被直接运行时才执行入口逻辑；被 import 时不执行。
history = []	保存完整上下文，是 agent 的短期记忆。
input(...)	终端读取用户输入。
query.strip().lower()	去空格、转小写，用于判断退出命令。
("q", "exit", "")	tuple，可换成 list；工程上用 set 更适合查找。
history[-1]	取对话历史最后一条消息。

tuple/list/set 的区别

("q", "exit", "") 是不可变 tuple；["q", "exit", ""] 是可变 list；{"q", "exit", ""} 是 set，查找语义更直接，工程上常用于退出命令集合。

8. Block / getattr / block.text 的理解

Claude 风格的 response.content 不是普通字符串，而是一组结构化输出块。

Block 类型	常用字段	用途
TextBlock	block.type == "text"; block.text	模型要展示给人的自然语言文本。
ToolUseBlock	block.type == "tool_use"; block.input; block.id	模型请求调用某个工具。
tool_result	type/content/tool_use_id	你执行工具后构造的结果，喂回模型。

```
for block in response_content:
    if getattr(block, "type", None) == "text":
        print(block.text)
```

为什么不用 print(block.content)

因为 TextBlock 没有 .content，文本在 .text；ToolUseBlock 也没有 .content，参数在 .input。必须先看 block.type 再决定访问哪个字段。

getattr 的作用

getattr(block, "type", None) 是安全访问属性：如果 block 没有 type 属性，不会崩溃，而是返回 None。

9. 常见报错与定位

现象/报错	高概率原因	修复方向
Inconsistent use of tabs and spaces	同一代码块混用了 Tab 和空格缩进	VSCode Convert Indentation to Spaces，统一 4 spaces
NameError: os is not defined	使用 os.getenv/os.getcwd 前忘记 import os	文件顶部加 import os
SYSTEM 变成 tuple	字符串末尾多了逗号：SYSTEM = f"...",	去掉末尾逗号
Could not resolve authentication method	SDK 没拿到 api_key/auth_token/credentials	不要 pop ANTHROPIC_API_KEY；显式传 api_key
工具执行了但没最终回答	agent_loop 少了 while True	工具结果 append 后继续 messages.create
模型自称 Claude	使用 Anthropic-compatible 协议且 system 未覆盖身份	在 system 中明确 provider/model 身份，或接受其只是身份错觉

调试顺序建议

先看 Python 语法错误，再看环境变量是否加载，再看 client 初始化是否有 key/base_url，再看 stop_reason 和 response.content 结构，最后检查 tool_result 是否回传。

10. SDK 封装层与结构化 Block 返回



思维导图 4: Anthropic SDK 的封装层

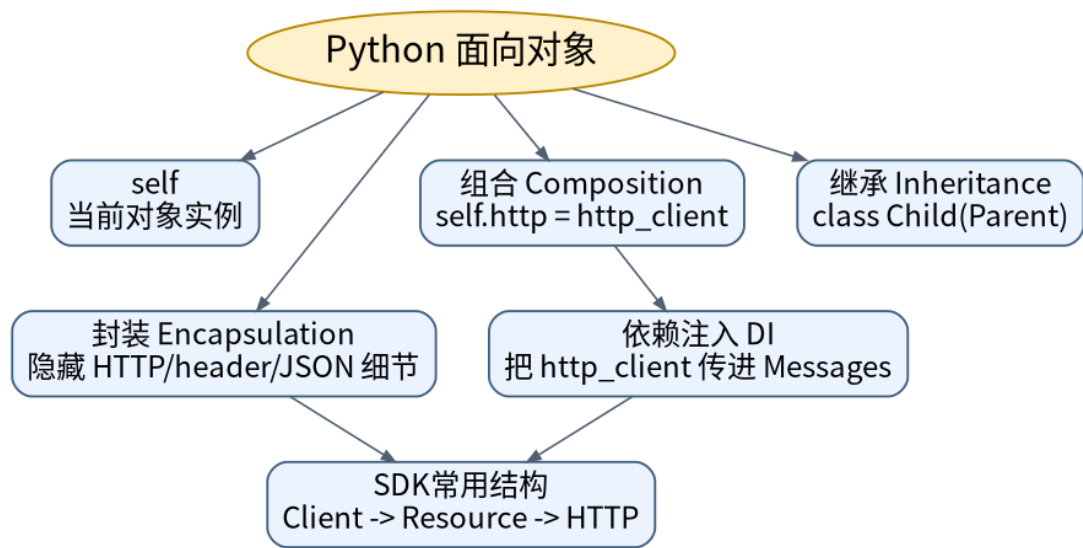
Anthropic Python SDK 的本质是：把 HTTP API 封装成 Python 方法调用，并把 API 返回的 JSON 转换成 Python 对象。

```
# 你写的是：
response = client.messages.create(...)

# SDK 内部近似做的是：
POST /v1/messages
raw_json = {...}
response.content = parse_blocks(raw_json["content"])
```

层级	作用
Client 层	保存 base_url、api_key、headers、http client 等通用配置。
Resource 层	messages.create 是 messages 资源的 create 方法。
HTTP 层	真正负责 POST、重试、超时、错误处理。
Model/Parser 层	把 JSON 中 type=text/tool_use 的条目转成 TextBlock/ToolUseBlock。

11. Python 封装、组合、继承与 self.http



思维导图 5: Python OOP 中 self、封装、组合与继承

```
class Messages:
    def __init__(self, http_client):
        self.http = http_client

    def create(self, model, messages, tools=None):
        return self.http.post("/v1/messages", json={...})
```

概念	解释	例子
self	当前对象实例本身	self.http 等价于 “这个 Messages 对象的 http 属性”
封装	把复杂细节藏在类内部，对外只暴露简单方法	client.messages.create() 隐藏 HTTP/header/JSON 细节
组合	一个对象持有另一个对象作为属性	self.http = http_client
继承	子类复用父类属性和方法	class RetryHTTPClient(HTTPClient): ...
依赖注入	把依赖从外部传入，而不是在类里面写死	Messages(http_client)

重点：self.http 不是继承

self.http = http_client 是组合。它的好处是 HTTP 层可以替换成 MockHTTP、RetryHTTPClient、StreamingHTTPClient，资源层 Messages 不需要改。

12. 推荐修正版与工程化方向

保留 Anthropic block 体验，同时接 DeepSeek 兼容接口时，建议写得更显式：

```
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv(override=True)

client = Anthropic(
    api_key=os.getenv("DEEPSEEK_API_KEY") or os.getenv("ANTHROPIC_API_KEY"),
    base_url=os.getenv("ANTHROPIC_BASE_URL"),
)
MODEL = os.environ["MODEL_ID"]
```

如果后续要支持更多模型，不建议把 agent runtime 绑死在某个 SDK 上，可以做一个 adapter 层：

```
class BlockLLM:
    def create(self, messages, tools):
        """统一返回：Response(content=[TextBlock/ToolUseBlock], stop_reason=...)"""
        raise NotImplementedError

class AnthropicAdapter(BlockLLM):
    ...

class OpenAICompatibleAdapter(BlockLLM):
    ... # 把 OpenAI/DeepSeek 的 tool_calls 转成统一 ToolUseBlock
```

工程方向	说明
统一 Block Adapter	让 agent_loop 永远只处理 TextBlock/ToolUseBlock，不关心底层 provider。
Tool Router	不只 bash，可加入 python、file、search、git 等工具。
Sandbox	隔离命令执行环境，降低 shell=True 风险。
Streaming	边生成边显示，边执行工具事件。
Tracing/Logging	记录每次 tool_use、tool_result、token/cost、错误堆栈。

13. 快速复习清单

- 看到 `client.messages.create`，就想到：Anthropic 风格 Messages API + block 返回。
- 看到 `client.chat.completions.create`，就想到：OpenAI 风格 Chat Completions API。
- 看到 `ANTHROPIC_BASE_URL=https://api.deepseek.com/anthropic`，就想到：DeepSeek 的 Anthropic-compatible 兼容层。
- `TOOLS` 是给模型看的工具说明书，不是真正执行器。
- `run_bash` 才是真正执行 shell 命令的函数。
- `response.content` 是 blocks 列表，不是普通字符串。
- `TextBlock` 读 `block.text`；`ToolUseBlock` 读 `block.input`。
- `tool_result` 必须带 `tool_use_id`，模型才能知道结果对应哪次调用。
- `agent_loop` 必须循环：工具结果要再喂回模型。
- `self.http` 是组合/依赖注入，不是继承。

最终大图景

你这次学习的核心不是“某个 SDK 怎么写”，而是如何把 LLM、工具调用、执行器、环境变量、SDK 封装和 Python OOP 组合成一个最小可运行的 AI Coding Agent。